

Object Lifeline

Object lifetime refers to the entire duration that an object exists in the computer's memory, from the exact moment it is created to the exact moment it is destroyed.

If an object dies too early, your program crashes (**dangling pointer**). If it lives too long, your program eats up memory (**memory leak**).

An object's lifetime generally falls into one of 4 categories:

1. Automatic Lifetime (The Stack)

Objects created within a **function** or **block { }**.

- **Birth:** The object is born exactly when the execution reaches the line of code that defines it.
- **Death:** It dies automatically the second the execution reaches the **closing brace }** of that scope.

```
void playFetch() {  
    Car myCar("Toyota"); // myCar is born here! (Stored on the Stack)  
    myCar.drive();  
} // myCar is instantly and automatically destroyed here.
```

2. Dynamic Lifetime (The Heap)

Objects created using the **"new"** keyword.

- **Birth:** Created exactly when you call **"new"**.
- **Death:** Destroyed exactly when you call **"delete"**.

```
void buyCar() {  
    Car *myCar = new Car("Toyota"); // Born here! (Stored on the Heap)  
} // Scope ends, but the Car still exists in memory!  
  
delete myCar; // Now it finally dies.
```

3. Static Lifetime (Global)

Objects that exist for the entire duration of the program.

- **Birth:** Created when the program first starts up, before the **main()** function even runs.
- **Death:** Destroyed when the program completely shuts down.
- **Example:** Global variables, or variables marked with the **static** keyword.

```
static Car factoryCar; // Lives for the entire duration of the program
```

4. Temporary Lifetime

Unnamed objects created during **expression evaluation**.

- **Birth:** Mid-expression (e.g., passing a temporary object to a function).
- **Death:** Immediately after the expression is evaluated.

```
processCar(Car("Temp")); // Created and destroyed in this single line
```

The Lifecycle Bookends: Constructors and Destructors

Every time an object is born and dies, a specific sequence of events happens. These are the **"bookends"** of an object's life.

1. The Constructor (Birth): Its job is to initialize the object and make it ready to use.

2. The Destructor (Death): Its job is to clean up any messes the object made during its lifetime (like closing open files, shutting down network connections, or deleting other objects it created on the Heap).

RAII

Because managing dynamic lifetime (Heap) is dangerous, modern C++ relies on a design pattern called **RAII - Resource Acquisition Is Initialization** (See RAII for details).